

ONBRD-99: Best Practice Summary

Series: ONBRD — Dynatrace Onboarding | **Reference:** 99 — Best Practice Summary |
Created: March 2026 | **Last Updated:** 05/06/2026

Overview

This notebook consolidates every actionable best practice from the ONBRD series (notebooks 01-10) into a single reference. Each practice is definitive: it states exactly what to set and why. Use this as a checklist when onboarding a new Dynatrace environment.

Where to go deeper after ONBRD: the topic series listed at the bottom (Section 11) cover each domain in depth — IAM, ORGNZ, FAQ, OPLOGS/OPMIG/OPIPE, K8S, CLOUD, OTEL, AIOPS, WFLOW, DASH, AUTOM, MZ2POL, ADOPT, plus per-platform migration series.

Table of Contents

1. [Environment Setup and Navigation](#)
 2. [IAM and Authentication](#)
 3. [ActiveGate Deployment](#)
 4. [Cloud and SaaS Integrations](#)
 5. [OneAgent Deployment](#)
 6. [Kubernetes Deployment](#)
 7. [Organization: Tags, Segments, and Naming](#)
 8. [Data Exploration and DQL](#)
 9. [Alerting and Workflows](#)
 10. [Dashboards and Visualization](#)
 11. [Where to Go Deeper — Topic Series Map](#)
-

1. Environment Setup and Navigation

Source: ONBRD-01

Practice	Recommended Setting/Value	Priority
Record your tenant ID	Write down the tenant ID from <code>https://{tenant-id}.apps.dynatrace.com</code> on day one	Critical
Bookmark tenant URL	Save <code>https://{tenant-id}.apps.dynatrace.com</code> in browser bookmarks	Critical
Use quick search for navigation	Press Cmd+K (Mac) or Ctrl+K (Windows/Linux) to find anything	Recommended
Pin frequently used apps	Pin Hosts, Problems, Logs & Events, Notebooks, and Workflows to the sidebar	Recommended
Use Platform Tokens (<code>dt0s16 / dt0s01</code>) for new automation	Sprint-1.337 default; reserve OAuth for external integrations and Classic API tokens (<code>dt0c01</code>) for legacy / installer downloads	Critical
Run discovery queries before deploying	Execute <code>smartscapeNodes "HOST" \ summarize count() (modern)</code> or <code>fetch dt.entity.host \ summarize count() (legacy)</code> to check for existing data before any agent deployment	Recommended
Explore the Dynatrace Hub	Install apps from the Hub to extend functionality before building custom solutions	Optional

2. IAM and Authentication

Source: ONBRD-02 → IAM series for depth

Practice	Recommended Setting/Value	Priority
Configure IAM before deploying agents	Set up SAML/SSO and groups before inviting users or deploying OneAgent	Critical
Enable SAML 2.0 or OIDC SSO	Configure enterprise SSO via Account Management > Identity & access management > Single sign-on	Critical
Set Name ID Format to email	In your IdP SAML app, set Name ID Format = Email address	Critical

Practice	Recommended Setting/Value	Priority
Map required SAML attributes	Map <code>email</code> , <code>firstName</code> , <code>lastName</code> from your IdP to Dynatrace	Critical
Keep a break-glass local admin	Maintain at least one local admin account with documented credentials in case SSO fails	Critical
Create four standard user groups	Platform Admins (Admin), SRE Team (Configurator), Developers (Operator), Stakeholders (Viewer)	Critical
Link IdP groups to Dynatrace groups	Map IdP group membership to Dynatrace groups for automatic provisioning/deprovisioning	Recommended
Use policies for access control	Use environment policies, account policies, and data policies (with Segments + <code>dt.security_context</code>) instead of legacy Management Zones	Critical
Use parameterized policies (Sprint-1.337+ pattern)	Prefer one policy with <code>\${bindParam:NAME}</code> bound to multiple groups via binding parameters over many copies of the same policy with hardcoded scope values	Critical
Standardize on <code>dt.security_context</code> as the boundary field	Tag all data and configurations with <code>dt.security_context</code> ; bind IAM policies to it. Without it, cross-entity-type policies cannot be written	Critical
Use Platform Tokens for new automation	<code>dt0s16</code> / <code>dt0s01</code> is the recommended default (sprint-1.337); use <code>Authorization: Bearer <token></code> scheme	Critical
Use OAuth Clients for external integrations only	Reserve OAuth for SaaS-to-SaaS / account-admin automation that operates above the tenant level	Recommended
Use Classic API Tokens for legacy / installer downloads only	<code>dt0c01</code> with <code>Authorization: Api-Token <token></code> ; migrate to Platform Token where possible	Recommended
Use minimal token scopes	Grant only the scopes / policies each token needs; never create all-scope tokens	Critical
Name tokens descriptively	Use pattern <code>{env}-{purpose}</code> (e.g., <code>prod-oneagent-deployment</code> , <code>aws-activegate-useast1</code>)	Recommended

Practice	Recommended Setting/Value	Priority
Set token expiration to 90-365 days	Set explicit expiration on every API token to force rotation	Recommended
One token per use case	Create separate tokens per integration so you can revoke one without affecting others	Recommended
Never commit tokens to code	Store tokens in environment variables or secrets managers only	Critical
Test SSO in incognito window	Verify SSO in a private browser before enabling for all users	Critical
Review AG token schema before sprint-1.338 upgrades	Sprint-1.338 changes the ActiveGate token schema — review upgrade-notes for any AG-token-issuing automation	Recommended

3. ActiveGate Deployment

Source: ONBRD-03 → CLOUD series for cloud-specific deep dives

Practice	Recommended Setting/Value	Priority
Deploy ActiveGate for restricted networks	Required when hosts cannot reach <code>*.dynatrace.com</code> on port 443 directly	Critical
Deploy ActiveGate for Extensions (3rd-gen and 2.0)	Required for SNMP, database extensions, VMware, and custom data sources	Critical
Deploy ActiveGate for private synthetic	Required for monitoring internal applications with synthetic tests	Critical
Deploy ActiveGate for GCP integration	Required — Clouds app GCP support is not yet available . AWS (Clouds-app GA) and Azure (Clouds-app preview) can use direct connections without AG	Critical
Deploy 2+ ActiveGates per zone in production	Set replicas: 2 minimum for HA; OneAgents auto-failover between AGs	Critical
Size ActiveGate by agent count	Small (<=500 agents): 2 GB / 2 cores. Medium (500-1500): 4 GB / 4 cores. Large (1500-3000): 8 GB / 8 cores	Critical

Practice	Recommended Setting/Value	Priority
Mount <code>/opt/dynatrace</code> on dedicated partition	Allocate 20 GB minimum for production AG installations	Recommended
Use SSD for extensions workloads	Set disk type to SSD when running extensions on the ActiveGate	Recommended
Place ActiveGate in same network zone as agents	AG must be in the same zone as the OneAgents it serves, with outbound HTTPS to <code>*.dynatrace.com</code>	Critical
Assign network zones during install	Use <code>--set-network-zone=<zone></code> parameter during AG installation	Recommended
Name PaaS tokens descriptively	Use <code>{env}-activegate-{zone}</code> pattern (e.g., <code>prod-activegate-dmz</code>)	Recommended
Use Dynatrace Operator for K8s ActiveGate	Deploy via DynaKube CR with <code>apiVersion: dynatrace.com/v1beta5</code> or <code>v1beta6</code>	Recommended
Use correct OpenShift image name	Sprint-1.338 fix: <code>dynatrace/dynatrace-activegate:<version></code> (not <code>dynatrace/activegate</code>)	Critical
Set pod anti-affinity for K8s AG	Use <code>requiredDuringSchedulingIgnoredDuringExecution</code> with <code>topologyKey: kubernetes.io/hostname</code>	Critical
Create PodDisruptionBudget for K8s AG	Set <code>minAvailable: 1</code> to prevent all replicas being evicted simultaneously	Critical
Set resource requests and limits for K8s AG	Small: requests 500m/1Gi, limits 2000m/2Gi. Scale per sizing table	Critical
Use ClusterIP for in-cluster routing	Only use LoadBalancer if external hosts must route through K8s-hosted AG	Recommended
Verify AG health after deployment	Check <code>curl -k https://localhost:9999/communication/health</code> and confirm "Connected" in Deployment Status > ActiveGates	Critical

4. Cloud and SaaS Integrations

Source: ONBRD-04 → CLOUD series for per-provider deep dives, AUTOM for extension automation

Practice	Recommended Setting/Value	Priority
Configure cloud integrations before OneAgent	Set up AWS/Azure/GCP integration first so Smartscape topology includes cloud services from day one	Recommended
Use Clouds app where supported	AWS (GA) and Azure (preview) — direct agentless connection. GCP not yet available in Clouds app, still requires AG-based polling	Recommended
Prefer Extensions 3rd-gen for new integrations	Sprint-1.337+ default for net-new extensions; Extensions 2.0 still supported for existing installs	Critical
Use IAM Role for AWS (not access keys)	Create an IAM role with CloudWatch read permissions and a trust relationship for Dynatrace's AWS account	Critical
Grant least-privilege cloud permissions	AWS: <code>cloudwatch:GetMetricData</code> , <code>tag:GetResources</code> , <code>ec2:DescribeInstances</code> , etc. Azure: Reader role. GCP: Monitoring Viewer role	Critical
Use Lambda DT_TAGS for tag propagation	Sprint-1.337 — set <code>DT_TAGS</code> env var on Lambda functions to propagate observability tags from the source	Recommended
Set AWS polling interval to 5 minutes	Use default 5-minute polling; do not set shorter intervals unless specifically needed	Recommended
Limit AWS regions to those with resources	Only enable regions where you have active workloads to avoid unnecessary API calls	Recommended
Use resource tags to filter monitored resources	In AWS/Azure/GCP settings, use tags to include/exclude resources from monitoring	Recommended
Install extensions from the Hub first	Check Dynatrace Hub for pre-built extensions before building custom ones	Recommended
Assign extensions to an ActiveGate group	Every extension instance must be assigned to an AG group for execution	Critical

Practice	Recommended Setting/Value	Priority
Use OpenTelemetry for direct ingest	For OTel-instrumented apps, send directly to Dynatrace (no ActiveGate required)	Recommended
Verify integration data with entity queries	Run <code>smartscapeNodes "AWS_LAMBDA_FUNCTION"</code> (modern) or <code>fetch dt.entity.aws_lambda_function</code> (legacy) to confirm entities are discovered	Critical

5. OneAgent Deployment

Source: ONBRD-05 → K8S series for K8s deployment depth

Practice	Recommended Setting/Value	Priority
Use phased rollout	Phase 1: 2-5 non-critical hosts (pilot). Phase 2: one application tier. Phase 3: full production	Critical
Generate dedicated deployment tokens	Create token with <code>InstallerDownload</code> scope named <code>{env}-oneagent-deployment</code>	Critical
Set host group at install time	Pass <code>--set-host-group=<group-name></code> (e.g., <code>prod-app-payments</code>); never retrofit later	Critical
Set primary tags at install time (Sprint-1.337+)	Use <code>oneagentctl --set-host-tag</code> to emit <code>environment</code> , <code>team</code> , <code>cost_center</code> , and <code>dt.security_context</code> as primary tags at the source	Critical
Set host properties during installation	Pass <code>--set-host-property=env=production --set-host-property=team=platform</code> at install time (legacy property model — primary tags are preferred)	Recommended
Set custom host display name	Use <code>--set-host-name="prod-web-01"</code> during installation for meaningful names	Recommended
Plan Windows Npcap install (Sprint-1.338)	New OneAgent Windows builds use Npcap (replaces WinPcap); plan Npcap install / golden-image distribution before rollout	Critical

Practice	Recommended Setting/Value	Priority
Restart existing processes after install	Restart application servers (Tomcat, JBoss, IIS), web servers, and app runtimes for full code-level visibility	Critical
Enable auto-update	Leave OneAgent auto-update enabled (default) to stay on supported versions	Recommended
Verify deployment with DQL	Run <code>timeseries cpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.smartscape.host} \ summarize active_hosts = count()</code> and confirm hosts appear within 5 minutes	Critical
Run connection check on each host	Execute <code>sudo /opt/dynatrace/oneagent/agent/tools/oneagent-connection-check</code> after installation	Recommended
Wait 15-30 minutes for full discovery	Allow time for topology, services, and dependencies to fully populate before assessing coverage	Recommended
Use Dynatrace Operator for all K8s deployments	Never install OneAgent manually on K8s nodes; use the Operator exclusively	Critical

6. Kubernetes Deployment

Source: ONBRD-03, ONBRD-05

Practice	Recommended Setting/Value	Priority
Use Cloud Native FullStack mode	Set <code>oneAgent.cloudNativeFullStack</code> in DynaKube CR; this is the recommended mode for all modern K8s	Critical
Use DynaKube API v1beta5 or v1beta6	Set <code>apiVersion: dynatrace.com/v1beta5</code> (or <code>v1beta6</code>); <code>v1beta3</code> is removed in Operator 1.8.0	Critical
Install Operator via Helm	Use <code>helm install dynatrace-operator dynatrace/dynatrace-operator --namespace dynatrace --set installCRD=true</code>	Recommended

Practice	Recommended Setting/Value	Priority
Create dedicated <code>dynatrace</code> namespace	All Dynatrace components go in the <code>dynatrace</code> namespace	Critical
Store tokens in K8s Secrets	Create secret <code>dynakube</code> with <code>apiToken</code> and <code>paasToken</code> (or <code>dataIngestToken</code>) keys	Critical
Enable <code>kubernetes-monitoring</code> capability on AG	Add <code>kubernetes-monitoring</code> to <code>activeGate.capabilities</code> for cluster API access	Critical
Enable <code>routing</code> capability on AG	Add <code>routing</code> to <code>activeGate.capabilities</code> for OneAgent traffic proxying	Critical
Use namespace selectors for multi-tenant clusters	Set <code>namespaceSelector.matchLabels</code> to isolate monitoring per team/tenant	Recommended
Mirror images for air-gapped environments	Pull <code>dynatrace/dynatrace-operator</code> , <code>dynatrace/dynatrace-oneagent</code> , <code>dynatrace/dynatrace-activegate</code> to private registry	Critical
Set tolerations for master/control-plane nodes	Add toleration for <code>node-role.kubernetes.io/master</code> with <code>operator: Exists</code>	Recommended
Use <code>--set-host-group</code> for cluster identification	Pass <code>args: ["--set-host-group=my-k8s-cluster"]</code> in <code>cloudNativeFullStack</code> spec	Recommended

7. Organization: Tags, Segments, and Naming

Source: ONBRD-06 → ORGNZ series, FAQ-01 (host group naming), FAQ-02 (tagging strategy), IAM series

Practice	Recommended Setting/Value	Priority
Tag at the source	Set host properties, OneAgent primary tags (sprint-1.337+), cloud tags, K8s labels, and OTEL attributes at the origin; do not rely on Dynatrace-side rule engines	Critical

Practice	Recommended Setting/Value	Priority
Standardize on <code>dt.security_context</code> as the boundary field	Decide value space first; emit on all signals via OneAgent primary tags or OpenPipeline enrichment; bind IAM policies to it	Critical
Define five standard host properties / primary tags	Set <code>environment</code> , <code>team</code> , <code>app</code> , <code>cost_center</code> , and <code>tier</code> on every OneAgent installation	Critical
Use lowercase, hyphen-separated property keys	Use <code>cost-center=eng</code> not <code>COST_CENTER=eng</code> or <code>CostCenter=eng</code>	Recommended
Use consistent property values	Always <code>prod</code> (never <code>sometimes production</code>); always <code>dev</code> (never <code>development</code>)	Critical
Use Segments for data filtering	Create reusable DQL-based Segments in Observe and explore > Segments; do not use legacy Management Zones	Critical
Use <code>contains()</code> as a function call in DQL	Write <code>contains(service.name, "payment")</code> — never SQL-style <code>service.name contains "payment"</code>	Critical
Name segments descriptively	Use <code>Prod-Checkout-Team</code> not <code>Segment1</code>	Recommended
Establish host naming convention	Use pattern <code>{env}-{tier}-{seq}</code> (e.g., <code>prod-web-01</code>) and set via <code>--set-host-name</code>	Recommended
Plan host group taxonomy before first install	See FAQ-01 (host group naming strategy); group by ownership × environment, not by team alone	Critical
Plan tagging sources & standards before tagging	See FAQ-02 (tagging sources, standards, strategy); decide which source is canonical for each dimension	Critical
Leverage cloud tags automatically	AWS tags appear as <code>aws.tag.*</code> , Azure as <code>azure.tag.*</code> , GCP as <code>gcp.label.*</code> ; ensure cloud resources are tagged consistently	Recommended
Use K8s labels for container organization	Filter by <code>k8s.namespace.name</code> , <code>k8s.deployment.name</code> , and <code>k8s.pod.labels.*</code> in DQL	Recommended
Document naming conventions	Write down property key/value standards and share with all teams before deployment	Critical

Practice	Recommended Setting/Value	Priority
Plan for MZ retirement	MZ-on-calculated-metrics is on the May-2026 deprecation list; migrate to Policies + Segments per the MZ2POL series	Recommended

8. Data Exploration and DQL

Source: ONBRD-07, ONBRD-08 → OPLOGS / OPMIG / OPIPE / SPANS for data-domain depth

Practice	Recommended Setting/Value	Priority
Always specify a time range on fetch	Use <code>fetch logs, from:-1h</code> not bare <code>fetch logs</code> ; default 2h scans excessive data	Critical
Filter early in the pipeline	Place <code>filter</code> immediately after <code>fetch</code> ; never <code>sort</code> or <code>summarize</code> before filtering	Critical
Select fields early	Use <code>fields</code> or <code>fieldsKeep</code> after filtering to reduce memory; drop unneeded columns	Recommended
Use <code>==</code> for exact matches	Use <code>==</code> (not <code>~</code>) when the full value is known; pattern matching is slower	Recommended
Use <code>{"a", "b"}</code> array syntax	Use curly braces and double quotes; never <code>('a', 'b')</code> SQL-style	Critical
Use <code>contains()</code> as a function	<code>contains(field, "value")</code> — never SQL-style infix operator	Critical
Alias all aggregations	Write <code>summarize c = count()</code> then <code>sort c desc</code> ; bare <code>count()</code> cannot be referenced in <code>sort</code>	Critical
Use <code>timeseries</code> for metrics	Never <code>fetch dt.metrics</code> ; always <code>timeseries avg(dt.host.cpu.usage), from:-1h</code>	Critical
Use <code>makeTimeseries</code> for event → series	Convert event-based data to time-bucketed metrics: <code>fetch logs \ makeTimeseries c = count(), interval:5m</code>	Recommended
Convert timeseries arrays to scalars	Use <code>fieldsAdd val = arrayAvg(ts)</code> before <code>filter</code> or <code>sort</code> on timeseries results	Critical
Use duration arithmetic, not nanosecond constants	Write <code>duration / 1ms</code> , never <code>duration / 1ms</code> ; use <code>(event.end - timestamp) / 1m</code> , never <code>/ 1m</code>	Critical

Practice	Recommended Setting/Value	Priority
Use <code>event.status / event.end</code> for Davis problems	<code>fetch dt.davis.problems</code> uses <code>event.status</code> (<code>ACTIVE / CLOSED</code>) and <code>event.end</code> — not <code>status / end_time</code>	Critical
Use <code>smartscapeNodes</code> for new entity queries	Modern: <code>smartscapeNodes "HOST"</code> ; legacy <code>dt.entity.host</code> still works on hybrid tenants but is deprecated	Recommended
Check for null before aggregating	Add filter <code>isNotNull(field)</code> before <code>summarize</code> on optional fields	Recommended
Use <code>entityName()</code> for display names	Resolve entity IDs to names with <code>entityName()</code> or <code>getNodeName(dt.smartscape.<type>)</code>	Recommended
Run discovery queries after deployment	Execute entity counts, service discovery, and log volume queries within 30 minutes of deployment	Recommended
Know the Grail data types	Logs (<code>fetch logs</code>), Spans (<code>fetch spans</code>), Metrics (<code>timeseries</code>), Events (<code>fetch events</code>), Bizevents (<code>fetch bizevents</code>), Problems (<code>fetch dt.davis.problems</code>), Security (<code>fetch securityEvents</code>), RUM (<code>fetch usersessions / user.events</code>)	Recommended

9. Alerting and Workflows

Source: ONBRD-09 → AIOPS (Davis depth), WFLOW (Workflows depth)

Practice	Recommended Setting/Value	Priority
Use Workflows for all alerting	Configure alerts in Automate > Workflows; do not use legacy Alerting Profiles	Critical
Create a detected problem trigger workflow first	Set trigger to "detected problem" with event = problem opens; this is the minimum viable alerting setup	Critical
Route alerts by team using conditions	Use JavaScript conditions like <code>event["affected_entity_ids"].some(id => id.includes("checkout"))</code> to send to team-specific channels	Critical

Practice	Recommended Setting/Value	Priority
Create one workflow per team/routing need	Separate workflows per team for independent maintenance and condition tuning	Recommended
Connect Slack via OAuth first	Go to Settings > Integration > Slack and complete OAuth before adding Slack actions to workflows	Recommended
Set up PagerDuty for critical path	Configure PagerDuty integration key in Settings > Integration > PagerDuty for on-call paging	Critical
Use Jinja2 templates in messages	Include <code>{{ event["title"] }}</code> , <code>{{ event["event.category"] }}</code> , and <code>{{ event["problem_url"] }}</code> for dynamic context	Recommended
Configure both open and close notifications	Add triggers for problem opens AND problem closes so teams know when issues resolve	Recommended
Test every workflow before relying on it	Use the workflow test feature to verify delivery, routing, and formatting	Critical
Monitor workflow execution history	Check Automate > Workflows > Executions regularly for failed runs	Recommended
Document escalation procedures	Write down who gets paged, when to escalate, and how to acknowledge across all teams	Critical
Review Davis sensitivity defaults	The five anomaly-detection mechanisms (static / auto-adaptive / seasonal / multi-dimensional baseline / novelty/forecast) each have tunable defaults — review per workload class	Recommended
Use Davis CoPilot / Dynatrace Assist for triage	Davis CoPilot provides DQL2NL, prompt-based investigation, and chat surfaces alongside Workflows	Recommended
Use Workflow AI tasks for agentic remediation	Workflow AI tasks + the MCP server enable agentic remediation flows (see WFLOW + AIOPS series)	Recommended

10. Dashboards and Visualization

Source: ONBRD-10 → DASH series for dashboard strategy and executive reporting depth

Practice	Recommended Setting/Value	Priority
Use Dashboards (new) for new work	Build on Dashboards (new) — Classic Dashboards are legacy. Dashboards (new) auto-refresh for NOC/status screens; Notebooks remain for ad-hoc investigation and RCA	Critical
Build an Executive Summary dashboard first	Include KPI row (uptime, errors, avg response, active problems), trend chart, problem list, service health tiles	Critical
Use Single Value tiles for KPIs	Display request count, error rate, host count, active problem count as single-value tiles	Recommended
Use SLO / SLI tiles where service contracts exist	SLO tiles communicate availability and latency targets at a glance	Recommended
Use <code>round(value, decimals: 2)</code> in dashboard queries	Round percentages and rates to 2 decimal places for clean display	Recommended
Set a default Segment on every dashboard	Apply an environment or team segment so viewers see only relevant data	Recommended
Set auto-refresh interval	Configure refresh rate (30s for NOC, 5m for status boards) in dashboard settings	Recommended
Create three standard dashboards	Executive Summary (KPIs + problems), Service Dashboard (requests, errors, P95, endpoints), Infrastructure Dashboard (host count, CPU, memory, top consumers)	Recommended
Match visualization to data type	Single number = Single Value. Trend = Line Chart. Distribution = Pie Chart. Ranked list = Top List. Entity health = Honeycomb	Recommended
Share dashboards with groups, not individuals	Set sharing to user groups (from ONBRD-02) for easier access management	Recommended
Export dashboards as JSON for backup	Use JSON export before major changes; store in version control	Optional

11. Where to Go Deeper — Topic Series Map

ONBRD covers the foundation. These topic series cover each domain in depth:

Domain	Topic Series	Notebooks
IAM administration	IAM	13
Data organization (buckets, segments, security context)	ORGNZ	11
Frequently asked questions	FAQ	growing collection
OpenPipeline log processing	OPLOGS	9
Classic Logs → OpenPipeline migration	OPMIG	10
OpenPipeline beyond logs (spans, metrics, events, bizevents)	OPIPE	7
Distributed tracing & spans	SPANS	9
Kubernetes monitoring & DynaKube	K8S	15
Cloud integration deep dives (AWS, Azure, GCP)	CLOUD	9
OpenTelemetry integration	OTEL	9
Workflows & alert notifications	WFLOW	10
Dynatrace Intelligence (Causal/Predictive/Generative AI, Davis)	AIOPS	8
Dashboard strategy & executive reporting	DASH	8
Configuration automation & GitOps	AUTOM	11
Management Zone → Policy migration	MZ2POL	10
Web Real User Monitoring	WEBRUM	9
Native mobile monitoring	MOBL	13
Synthetic monitoring	SYNTH	7
Business events & funnel analytics	BIZEV	7
Database monitoring	DBMON	7
Platform maturity & adoption roadmap	ADOPT	6

Domain	Topic Series	Notebooks
Managed → SaaS migration	M2S	10
SaaS → SaaS migration	S2S	11
New Relic → Dynatrace (procedural runbook)	NR2DT	11
New Relic → Dynatrace (component deep dives)	NRLC	9
Sumo Logic → Dynatrace	SL2DT	10
Splunk → Dynatrace	S2D	10

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.